

Paralelismo com Memória Compartilhada

Caick Wendell Lopes dos Santos¹, Camila De Araújo Bastos²

¹Departamento de Tecnologia – Universidade Estadual de Feira de Santana (UEFS)
Av. Transnordestina, s/n, Novo Horizonte / Feira de Santana – BA, Brasil- 44036-9

1. Introdução

Este relatório técnico apresenta uma análise de desempenho e escalabilidade de diferentes abordagens de implementação do Conway's Game of Life (Jogo da Vida). O algoritmo consiste em um autômato celular bidimensional regido por regras de sobrevivência, morte e reprodução das células, determinadas pelo estado de sua vizinhança de Moore, composta pelas oito posições adjacentes de cada célula, formando uma vizinhança 3×3 .

A estrutura de dados utilizada representa o universo por meio de uma matriz toroidal, na qual as bordas da matriz são conectadas continuamente por meio de operações de módulo aritmético. Essa abordagem permite simular um universo sem bordas, garantindo que as células localizadas nas extremidades também possuam uma vizinhança completa.

A evolução do sistema é realizada de forma síncrona, utilizando matrizes de estado temporárias. Dessa maneira, o novo estado de cada célula é calculado a partir do estado completo da geração anterior, evitando que células já atualizadas interfiram no cálculo das demais células da mesma geração.

2. Objetivo

A análise busca comparar as diferenças de tempo de execução, *speedup*, eficiência paralela, comportamento da hierarquia de memória e custos associados ao paralelismo em cada abordagem. Também são comparados os resultados das implementações em C, uma linguagem compilada, e Python, uma linguagem interpretada, considerando as diferenças de desempenho e os efeitos dessas características na execução do algoritmo em arquiteturas multicore.

Além disso, pretende-se avaliar o impacto do Global Interpreter Lock (GIL) na implementação com *multithreading* em Python, considerando sua influência sobre a execução concorrente de operações intensivas em CPU. A análise também compara essa abordagem com o *multiprocessing*, que utiliza processos independentes e permite explorar múltiplos núcleos de processamento sem a mesma limitação do GIL.

Por fim, são analisadas as diferenças entre os mecanismos de comunicação utilizados nas implementações com *multiprocessing*, comparando a comunicação por Pipe, que envolve a transferência de dados entre espaços de memória independentes, com a utilização de memória compartilhada, que permite o acesso dos processos a uma região comum de memória e pode reduzir custos de cópia e serialização.

3. Metodologia e Implementação

Para avaliar o impacto de diferentes estratégias de paralelização sobre o desempenho do Jogo da Vida de Conway, foram implementadas seis variantes do algoritmo, todas derivadas de uma mesma especificação de problema e de um mesmo baseline serial.

- C serial;
- C paralelizado com OpenMP;
- Python serial;
- Python com multithreading utilizando ThreadPoolExecutor;
- Python com multiprocessing utilizando comunicação por Pipe;
- Python com multiprocessing utilizando memória compartilhada.

Todas as seis implementações compartilham a mesma condição inicial, as mesmas regras de transição e o mesmo critério de parada, garantindo a comparabilidade dos resultados. As versões diferem na linguagem utilizada, no modelo de execução e na forma como o trabalho é decomposto e sincronizado entre as unidades de execução concorrentes.

3.1. Definição do Problema e Parâmetros Comuns

Em todas as implementações, o universo é representado por uma grade toroidal (bordas conectadas por aritmética modular) de dimensões $W \times H$, configuráveis por variáveis de ambiente (`GOL_W`, `GOL_H`), com valor padrão de 500×500 células. O estado inicial é determinístico e idêntico em todas as versões: uma cruz que atravessa o tabuleiro pela linha e pela coluna centrais ($x = W/2$ ou $y = H/2$), o que garante que qualquer divergência de resultado entre implementações seja atribuível à lógica de paralelização e não a diferenças de inicialização. O número de gerações simuladas também é parametrizável (`GOL_ITER`, padrão 5000), executando-se sempre `max_iter + 1` passos de evolução para manter paridade entre todas as versões.

As regras de transição seguem a formulação clássica do Jogo da Vida: para cada célula, contam-se os vizinhos vivos na vizinhança de Moore (oito células adjacentes, com vizinhança toroidal); uma célula viva permanece viva se possuir 2 ou 3 vizinhos vivos, e uma célula morta nasce se possuir exatamente 3 vizinhos vivos; qualquer outra condição resulta em morte ou permanência do estado morto.

Nas versões em C, os parâmetros $w = 500$, $h = 500$ e `max_iter = 5000` são fixados diretamente no código-fonte (em `main`), ao passo que as versões em Python os leem de variáveis de ambiente com os mesmos valores como padrão — diferença que não afeta a comparabilidade dos resultados, uma vez que ambas as linguagens são executadas com a mesma configuração de tamanho de grade e número de gerações. Por fim, ao término de todas as iterações, o programa realiza a contagem total de células vivas no tabuleiro. Essa etapa final foi introduzida em todos os códigos estritamente para contornar as otimizações do compilador C (como a eliminação de código morto, ou *dead code elimination*): caso o estado final da matriz não fosse utilizado para produzir nenhuma saída, o compilador descartaria todo o processamento do laço principal, reduzindo o tempo de execução da simulação a zero.

3.2. Paralelização por Colunas

A estratégia de paralelização adotada baseou-se na divisão geométrica do tabuleiro em blocos de colunas, atendendo a um requisito explícito do trabalho. Do ponto de vista de arquitetura e desempenho, essa escolha traz implicações diretas para a localidade espacial de cache: como as matrizes em C e as listas nativas do Python adotam o armazenamento *row-major* (elementos contíguos dispostos linha por linha), o processamento restrito a fatias verticais estreitas impede o aproveitamento ideal das linhas de cache, o

que eleva consideravelmente a taxa de cache misses. Apesar desse impacto na hierarquia de memória, a topologia por colunas simplifica o padrão de comunicação, pois restringe as áreas de fronteira (as "colunas fantasmas") estritamente às bordas laterais direita e esquerda, facilitando a troca de mensagens e a sincronização entre os processos ou threads vizinhos.

3.3. Implementação Serial em C (`gol.c`)

A versão serial em C constitui o baseline de mais baixo nível do trabalho, servindo de referência tanto para a versão paralela em OpenMP quanto para as reimplementações em Python. O universo é representado por uma matriz `unsigned univ[h][w]` alocada na pilha, e a função `evolve` segue exatamente a mesma estrutura de quatro laços aninhados descrita na versão Python: para cada célula (x, y) , os vizinhos são contados por meio de dois laços internos que percorrem a vizinhança 3×3 com indexação toroidal via aritmética modular $((y1 + h) \% h, (x1 + w) \% w)$; o próprio valor da célula é descontado quando ela está viva, e o novo estado é escrito em uma matriz local `new[h][w]`, também alocada na pilha, evitando a leitura de estados já atualizados da mesma geração. Uma segunda varredura completa copia `new` de volta para `univ`, reproduzindo o mesmo esquema de *double buffering* explícito da versão Python — de fato, a implementação em Python de `gol.py` foi construída como uma tradução direta desta versão em C, célula a célula e laço a laço, para servir de baseline funcionalmente equivalente entre as duas linguagens.

3.4. Implementação Paralela em C com OpenMP (`gol_omp.c`)

A versão `gol_omp.c` paraleliza a fase de evolução do algoritmo por meio de diretivas `#pragma omp parallel for` aplicadas ao laço externo sobre o eixo x (colunas), tanto na etapa de cálculo do próximo estado quanto na etapa de cópia de volta para `univ` e na inicialização do tabuleiro. Note-se que, em relação à versão serial, os laços de `evolve` são deliberadamente invertidos — o laço externo percorre x e o laço interno percorre y — de modo que a divisão de iterações feita automaticamente pelo OpenMP entre as *threads* ocorra por **colunas** e não por linhas, replicando a mesma decomposição unidimensional empregada manualmente nas versões multithread e multiprocessing em Python.

Cada *thread* do OpenMP processa uma faixa contígua de colunas da matriz `new` de forma independente, sem necessidade de exclusão mútua, pois threads distintas nunca escrevem na mesma coluna. Como a matriz `univ` original permanece somente-leitura durante todo o cálculo da geração seguinte (a escrita ocorre exclusivamente em `new`), não há condição de corrida entre as *threads* na fase de cálculo; a barreira implícita ao final de cada região `parallel for` garante que todas as colunas de `new` estejam completas antes do início do laço de cópia paralela subsequente, que por sua vez também está protegido por uma barreira implícita antes do avanço para a próxima geração. Diferentemente das versões multithread e multiprocessing em Python, que precisam implementar manualmente a divisão de blocos, o balanceamento do resto da divisão e, no caso dos processos, a comunicação de bordas, o OpenMP delega ao *runtime* a criação das *threads*, a divisão do intervalo de iterações e a sincronização por barreiras implícitas, exigindo apenas a anotação dos laços paralelizáveis e a garantia de que os laços tenham iterações independentes.

3.5. Implementação Serial em Python (`gol.py`)

A versão serial estabelece o baseline de corretude e desempenho contra o qual as demais implementações são comparadas. A função `evolve` realiza, para cada geração, uma varredura completa da matriz $W \times H$: para cada célula, quatro laços aninhados percorrem a vizinhança 3×3 centrada na célula, contabilizando vizinhos vivos por meio de indexação modular $((y1 + h) \% h, (x1 + w) \% w)$, o que implementa a topologia toroidal sem necessidade de tratamento especial de bordas. O próprio valor da célula é descontado da contagem quando ela está viva, e o novo estado é escrito em uma matriz auxiliar (`new`), evitando que a leitura da vizinhança de uma célula seja corrompida pela escrita do novo estado de outra célula na mesma geração. Uma segunda passagem completa copia o estado calculado de volta para a matriz `univ`, caracterizando um esquema de *double buffering* explícito. A contagem final de células vivas (`count_alive`) é feita por soma sequencial simples, servindo de referência para validar a redução paralela nas demais versões.

3.6. Implementação com Multithreading em Python (`gol_multithread.py`)

Esta variante paraleliza a mesma lógica de evolução por meio de um `ThreadPoolExecutor`, decompondo o domínio em blocos contíguos de colunas — decomposição unidimensional análoga à empregada nas versões baseadas em processos. Como todas as *threads* compartilham o mesmo espaço de memória do processo, não há necessidade de comunicação explícita de bordas: cada *thread* lê diretamente das colunas vizinhas dentro da mesma matriz `univ`.

A evolução de cada geração é dividida em duas fases sincronizadas por barreiras implícitas de `Future.result()`:

- **Fase 1 (`process_chunk`):** cada *thread* calcula o próximo estado de seu bloco de colunas $[start_x, end_x)$ e escreve o resultado na matriz auxiliar `new`, sem jamais escrever em `univ`;
- **Fase 2 (`copy_chunk`):** somente após todas as *threads* da Fase 1 terminarem (garantido pelo bloqueio até a resolução de todos os `Futures`), cada *thread* copia seu bloco de `new` de volta para `univ`.

A separação estrita entre as duas fases evita condições de corrida: se a cópia ocorresse simultaneamente ao cálculo, uma *thread* poderia ler um estado parcialmente atualizado da geração seguinte ao processar a vizinhança de uma célula na borda de seu bloco. A contagem final de células vivas segue o mesmo princípio de decomposição por colunas, em que cada *thread* soma um bloco disjunto (`count_chunk`) e retorna o total parcial via valor de retorno do `Future`, evitando o uso de `Lock` para proteção de uma variável compartilhada — abordagem funcionalmente equivalente a uma cláusula `reduction(+:total)` do OpenMP.

Cabe notar que, por se tratar de paralelismo baseado em *threads* no CPython, o ganho de desempenho desta versão está sujeito às limitações impostas pelo *Global Interpreter Lock* (GIL), que serializa a execução de *bytecode* Python entre *threads*, restringindo o paralelismo efetivo majoritariamente aos períodos em que o interpretador libera o GIL.

3.7. Implementação com Multiprocessamento em Python e Troca de Bordas via Pipe (`gol_multiprocess.py`)

Esta versão contorna a limitação do GIL utilizando processos independentes (`multiprocessing`), cada um com seu próprio interpretador e espaço de memória. O domínio é decomposto em blocos contíguos de colunas (`_split_columns`), distribuindo o resto da divisão $W \bmod n_tasks$ entre os primeiros blocos para equilibrar a carga. Cada processo trabalhador (`_worker`) mantém localmente apenas seu bloco de colunas, o que, ao contrário da versão `multithread`, exige comunicação explícita entre processos vizinhos para obter as colunas de fronteira (`ghost columns`) necessárias ao cálculo da vizinhança nas bordas do bloco.

A topologia de comunicação é estabelecida por um anel de Pipes bidirecionais: cada processo mantém uma conexão com o vizinho à esquerda e outra com o vizinho à direita, formando uma cadeia circular fechada consistente com a topologia toroidal do tabuleiro (o último bloco se comunica com o primeiro). A cada geração, antes do cálculo, cada processo envia sua coluna mais à esquerda e mais à direita aos respectivos vizinhos e recebe as colunas de fronteira correspondentes (`left_ghost`, `right_ghost`) — um caso particular é tratado quando $n_tasks = 1$, em que o próprio bloco fornece suas colunas fantasma via indexação toroidal direta, sem uso de Pipe. O cálculo da vizinhança em cada célula do bloco consulta a matriz local para colunas internas e as colunas fantasma recebidas para colunas na fronteira do bloco.

Ao término das `max_iter + 1` gerações, cada processo soma localmente as células vivas de seu bloco — equivalente a uma cópia privada de acumulador, no espírito de uma redução OpenMP — e envia o total parcial ao processo principal por meio de um Pipe dedicado exclusivamente à contagem (`count_conn`), desacoplado do Pipe originalmente reservado à extração periódica do tabuleiro completo para geração de arquivos PBM. O processo principal recebe as somas parciais de todos os trabalhadores antes de sincronizar o encerramento (`join`), evitando bloqueios por saturação do buffer dos Pipes. O custo principal desta abordagem é a serialização (*pickling*) das colunas de fronteira a cada geração para transmissão entre processos, repetida n_tasks vezes por passo de simulação.

3.8. Implementação com Multiprocessamento em Python e Memória Compartilhada (`gol_multiprocess_shared_memory.py`)

Esta variante mantém a mesma decomposição lógica por blocos de colunas da versão anterior, porém substitui a troca de bordas via Pipe por leitura direta em um bloco único de memória compartilhada (`multiprocessing.shared_memory.SharedMemory`), eliminando a serialização e a cópia de dados a cada geração. O buffer compartilhado é dimensionado para armazenar dois planos completos do tabuleiro ($2 \times W \times H$ bytes), permitindo um esquema de *double buffering* global: os deslocamentos `cur_off` e `next_off` alternam entre os dois planos a cada geração, de modo que todos os processos leem do mesmo plano "atual" e escrevem no mesmo plano "seguinte" simultaneamente, sem necessidade de troca explícita de mensagens.

Como todos os processos enxergam o tabuleiro inteiro (e não apenas seu próprio bloco mais colunas fantasma), a leitura de vizinhança de fronteira é feita diretamente sobre o buffer compartilhado, com indexação toroidal via aritmética modular idêntica à

da versão serial. A sincronização entre processos — necessária para garantir que nenhum processo comece a ler o plano "seguinte" como "atual" antes que todos os demais tenham terminado de escrevê-lo — é feita por uma `multiprocessing.Barrier` com $n_tasks + 1$ participantes (os n_tasks trabalhadores mais o processo principal), que aguarda ao final de cada geração antes de permitir a alternância dos planos.

Uma otimização adicional presente apenas nesta versão é a substituição do cálculo de vizinhança célula a célula por uma tabela de consulta pré-computada (RULE), indexada por um valor composto que combina a contagem de vizinhos vivos (n , 0 a 8) com o estado atual da célula deslocado 4 bits à esquerda ($n + (buf[...]\ll 4)$), reduzindo a lógica condicional das regras de transição a uma única leitura de tabela por célula. A contagem final de células vivas é feita por uma soma direta sobre a região do buffer correspondente ao plano vigente após a última geração (`count_alive`), executada uma única vez pelo processo principal, sem necessidade de redução distribuída, já que todo o tabuleiro já reside em um único espaço de memória acessível.

3.9. Comparação das Estratégias de Paralelização

O Quadro 1 resume as principais diferenças estruturais entre as seis implementações.

Tabela 1. Comparação das estratégias de decomposição e sincronização

Implementação	Linguagem	Unidade	Decomposição	Sincronização	Memória
Serial	C	—	Nenhuma*	—	Única
OpenMP	C	Thread	Colunas	Barreira implícita	Compartilhada
Serial	Python	—	Nenhuma	—	Única
Multithread	Python	Thread	Colunas	<code>Future.result()</code>	Compartilhada (GIL)
Multiprocess (Pipe)	Python	Processo	Colunas	Pipe (anel)	Isolada por processo
Multiprocess (Shared Mem.)	Python	Processo	Colunas	Barrier	Compartilhada (shm)

Em todas as versões paralelas, seja em C ou em Python, a decomposição do domínio segue o mesmo princípio unidimensional por faixas contíguas de colunas, o que preserva a comparabilidade dos resultados entre linguagens e modelos de execução: as diferenças de desempenho observadas na Seção 4 podem ser atribuídas ao mecanismo de sincronização, ao modelo de memória e ao custo de execução da linguagem empregados, e não a diferenças na estratégia de particionamento do problema ou na lógica das regras de transição, que é idêntica em todas as seis implementações.

3.10. Arquitetura das CPUs utilizadas

- **CPU 1:** AMD Ryzen 5 7520U (4 Núcleos / 8 Threads)
 - **Memória Cache:** L1: 256 KiB / L2: 2 MiB / L3: 4 MiB
 - **Memória RAM:** 16 GB (2x 8GB) LPDDR5 Samsung @ 5500 MT/s (Dual-Channel)
 - **Sistema Operacional:** Ubuntu 26.04

- **Kernel Linux:** 7.0.0-27-generic
- **CPU 2:** Intel Core i5-1135G7 (4 Núcleos / 8 Threads)
 - **Memória Cache:** L1: 320 KiB / L2: 5MB / L3: 8MB
 - **Memória RAM:** 12 GB DDR4
 - **Sistema Operacional:** Ubuntu 26.04
 - **Kernel Linux:** 7.0.0-29-generic

4. Resultados

4.1. Análise Comparativa de Arquiteturas (CPU 1 vs. CPU 2)

A execução da simulação em dois ambientes de *hardware* distintos — a CPU 1 (AMD Ryzen 5 com memória LPDDR5 a 5500 MT/s) e a CPU 2 (Intel Core i5 com memória DDR4) — evidencia como as características físicas de cada arquitetura reagem aos diferentes modelos de programação.

4.1.1. Desempenho Bruto e Hierarquia de Cache (C e OpenMP)

Nas implementações nativas em C, a CPU 2 obteve vantagem expressiva. Na versão serial, concluiu em 22,10s contra 50,71s da CPU 1; com OpenMP (4 *threads*), registrou 7,90s frente a 13,18s. Essa superioridade deriva da maior capacidade de cache L3 da arquitetura Intel, otimizada para *footprints* de memória entre 4 MB e 8 MB. Como a paralelização por colunas destrói a localidade espacial em matrizes *row-major*, um último nível de cache (LLC) mais robusto amortece a penalidade de acesso à RAM e assegura uma maior taxa de Instruções por Ciclo (IPC).

4.1.2. Banda de Memória e Serialização (Python Multiprocessing)

Nas abordagens de múltiplos processos em Python, o cenário se inverte. Na versão via *Pipe* com 4 processos, a CPU 1 assumiu a dianteira e finalizou em 3m33s, superando os 4m59s da CPU 2. A comunicação por *Pipe* exige constante serialização e desserialização (*pickling*) das fronteiras da matriz. Neste cenário altamente dependente da largura de banda, a tecnologia LPDDR5 a 5500 MT/s confere à arquitetura Ryzen uma superioridade decisiva sobre a DDR4 no tráfego dos dados.

4.1.3. Limitações do SMT (Simultaneous Multithreading)

Ambos os processadores operam com 4 núcleos físicos e 8 *threads* lógicas. Os dados demonstram que o *speedup* $S(p)$ cresce linearmente até $p = 4$. Contudo, a transição para $p = 8$ provoca uma queda acentuada na eficiência global $E(p)$. Durante a execução, as *threads* lógicas virtuais competem intensamente pelas mesmas Unidades Lógicas e Aritméticas (ALUs) e pelas mesmas caches L1/L2 dos núcleos físicos. Consequentemente, o SMT entrega ganhos computacionais marginais, o que justifica a degradação da curva de escalabilidade de ambas as máquinas ao se atingir a lotação máxima de *threads*.

Tabela 2. Comparativo de Tempo e Recursos por Arquitetura

Versão do Programa	Wall-clock		% CPU		Max RSS (KB)		Page Faults (Maj/Min)		Ctx Switches (Vol/Inv)	
	CPU 1	CPU 2	CPU 1	CPU 2	CPU 1	CPU 2	CPU 1	CPU 2	CPU 1	CPU 2
C Serial	50.71s	22.10s	99%	99%	3.752	3.960	0 / 569	0 / 570	1 / 1.371	1 / 709
C OpenMP (4 Threads)	13.18s	7.90s	399%	399%	4.112	4.252	4 / 584	0 / 589	6 / 345	1 / 112
Python Serial	20m 59s	20m 01s	99%	99%	13.960	13.992	0 / 2.397.447	0 / 2.397.003	2 / 19.752	1 / 10.048
Python Thread (4 Threads)	22m 25s	19m 13s	100%	100%	18.600	18.852	2 / 3.611	0 / 3.167	1.32M / 21.841	884.100 / 7.288
Python MP Pipe (4 Proc.)	3m 33s	4m 59s	391%	387%	19.264	19.140	1 / 9.595	0 / 9.161	15.859 / 7.915	16.227 / 5.707
Python MP SHM (4 Proc.)	4m 20s	5m 30s	376%	372%	20.544	20.312	13 / 8.736	0 / 8.315	64.538 / 8.363	63.930 / 8.383

4.2. Análise de Comportamento: CPU-bound vs. I/O-bound

Para determinar se uma aplicação possui comportamento predominantemente voltado ao processamento (*CPU-bound*) ou a operações de entrada e saída (*I/O-bound*), analisam-se conjuntamente o **percentual de utilização da CPU**, a proporção entre **User time** e **System time**, além da incidência de *falhas de página* (*page faults*) e operações de leitura/escrita em disco.

Com base nos dados extraídos através do comando `/usr/bin/time -v`, todas as cinco implementações avaliadas caracterizam-se estritamente como **CPU-bound**, embora apresentem nuances e penalidades de desempenho distintas derivadas de suas arquiteturas de execução.

Justificativas por Versão

1. C Serial e C OpenMP (4 Threads):

- **Comportamento:** *CPU-bound* puro.
- **Justificativa:** Ambas as versões apresentam um tempo de execução em modo kernel (*System time*) praticamente desprezível e um percentual de utilização da CPU que satura os recursos disponíveis (99% na versão serial e 399% na versão paralela com 4 núcleos). O consumo de recursos é inteiramente direcionado ao cálculo matemático intensivo da Vizinhança de Moore sobre a matriz toroidal, sem esperas por I/O de disco ou rede.

2. Python Serial:

- **Comportamento:** *CPU-bound* (com sobrecarga de interpretação).
- **Justificativa:** O tempo em modo utilizador (*User time*) domina a execução enquanto o tempo de kernel permanece irrelevante. O uso da CPU mantém-se fixo em 99%. O fator limitante não é o acesso a dispositivos externos, mas sim o custo computacional imposto pelo interpretador CPython ao processar os laços aninhados e gerir objetos dinâmicos no *heap*.

3. Python Multithreading (ThreadPoolExecutor):

- **Comportamento:** *CPU-bound* (travado por concorrência / *Lock-bound*).
- **Justificativa:** Embora o programa utilize múltiplas threads para computar blocos do tabuleiro, o uso global da CPU permanece limitado a ~100%. O tempo de execução é consumido quase na totalidade em espaço de utilizador para gerir a concorrência. A aplicação sofre de intensa contenção interna devido ao *Global Interpreter Lock* (GIL) e a milhões de trocas de contexto voluntárias, caracterizando uma carga estritamente

voltada ao processamento, mas severamente estrangulada pelas travas de sincronização da linguagem.

4. Python Multiprocessing (Pipe e Shared Memory):

- **Comportamento:** *CPU-bound* com forte penalidade de comunicação (*Communication-bound*).
- **Justificativa:** Registram percentuais de uso de CPU elevados (próximos a 380% com 4 processos) e *System times* baixos. Contudo, a variante baseada em *Pipes* introduz um custo computacional massivo de serialização (*pickling*) e desserialização de dados a cada geração para atualizar as colunas de fronteira. Embora o perfil fundamental permaneça centrado no esforço da CPU, o desempenho degrada consideravelmente devido ao tráfego de dados entre os espaços de memória isolados dos processos, aproximando-se de um comportamento restrito por largura de banda (*communication-bound*). A variante por *Shared Memory* mitiga esse problema ao eliminar a serialização, recuperando a eficiência do processamento em núcleo.

4.3. Análise Comparativa: C vs. Python (Abordagem Serial)

4.3.1. Paradigmas de Execução: Linguagem Compilada vs. Interpretada

A disparidade abissal de desempenho observada entre as implementações seriais reflete, fundamentalmente, a diferença arquitetural entre os paradigmas de execução de linguagens compiladas e linguagens interpretadas. O C é uma linguagem de tipagem estática e nativamente compilada (*ahead-of-time*). Durante o processamento do Jogo da Vida, suas instruções binárias atuam diretamente sobre a memória via aritmética de ponteiros, extraindo máxima vazão computacional. Em contrapartida, o CPython atua como um interpretador de *bytecode* dinâmico. Para uma simulação de matriz densa que realiza milhões de varreduras em laços aninhados, esse modelo introduz um *overhead* massivo, pois cada acesso celular exige verificações de tipo em tempo real e chamadas ao gerenciador de memória (*garbage collector*).

Tabela 3. Contadores de Desempenho Físico e Profiling (Versão C Serial)

Métrica Exigida	CPU 1 (AMD Ryzen 5)	CPU 2 (Intel Core i5)
Cycles (Ciclos Totais)	212,4 Bilhões	91,36 Bilhões
Instructions (Instruções)	161,2 Bilhões	160,94 Bilhões
IPC (Instructions Per Cycle)	0,76	1,76
Cache-misses	136,3 Milhões (Geral) / 350,4 Milhões (L1)	322 Milhões (L1)
Branch-misses	0,31% (56,3 Milhões)	0,25% (Taxa)
Valgrind Callgrind: Total de Instr. (Ir)	160.752.337.678	162.290.210.118
Valgrind Cachegrind: D1mr (Misses)	155.867.977	10.106.438
Total de Syscalls (<i>strace</i>)	35 chamadas	34 chamadas
Syscall dominante (<i>strace</i>)	<i>execve</i>	<i>execve</i>

4.3.2. Comprovação Empírica pelo Profiling de Hardware (*perf stat*)

A comparação dos dados do *perf stat* entre as versões seriais da CPU 1 evidencia o abismo entre as duas linguagens computando o mesmo problema. A versão C concluiu a

execução em aproximadamente 50 segundos, consumindo 212 bilhões de ciclos da CPU e executando 161 bilhões de instruções reais de máquina.

Para realizar a exata mesma tarefa e entregar a mesma matriz de saída, o interpretador Python demorou quase 21 minutos, o que demandou absurdos 5,3 trilhões de ciclos da CPU e 17,5 trilhões de instruções. Isso significa que, para cada instrução útil que o programa em C executou, o Python precisou executar mais de 100 instruções de "gerenciamento" interno de objetos (*overhead* do interpretador) apenas para viabilizar a lógica matemática da Vizinhança de Moore.

Além do volume de instruções, o paradigma de objetos do Python destruiu a localidade de referência na hierarquia de memória. Enquanto a matriz em C sofreu cerca de 136 milhões de faltas de cache (*cache-misses*) totais, as listas nativas aninhadas do Python (*list of lists*) registraram 408 milhões de *cache-misses* e impressionantes 4,4 bilhões de faltas na cache L1 de dados (*L1-dcache-load-misses*). A CPU gastou grande parcela de seus ciclos travada aguardando o processamento de referências de objetos Python espalhados desordenadamente pela RAM.

4.3.3. Comportamento do Sistema Operacional (*strace* e Hotspots)

A análise do comportamento sistêmico confirma que as duas linguagens penalizaram o hardware de formas distintas. Através do *gprof*, observa-se que no C 100% do tempo de execução concentrou-se estritamente na função matemática de processamento da grade (*evolve*). As interações com o sistema operacional (*strace*) foram insignificantes: apenas 8 chamadas de alocação genérica (*mmap*) que custaram singelos 75 microssegundos no tempo total.

Por outro lado, o CPython precisou escalar agressivamente os recursos dinâmicos. O *strace* registrou mais de 80.000 chamadas à função *brk* — uma requisição direta ao Kernel do Linux para expandir ou contrair o limite de espaço de memória do processo (*heap limit*). Essas alocações constantes ocorrem porque, ao reescrever a matriz em cada uma das 5000 iterações na função *evolve* (que reteve 1577 segundos do *cProfile*), o Python destrói os números inteiros antigos e instancia milhares de novos objetos do tipo *int* a cada etapa de evolução, sobrecarregando tanto a ALU do processador quanto as tabelas de gerenciamento de memória do Kernel.

Análise C Serial

Tabela 4. C Serial: Métricas de Hardware via perf stat (CPU 1)

Métrica	Valor	% Eventos
Cycles	212.423.047.978	71.42%
Instructions	161.218.929.814	71.43%
Cache-references	621.046.840	71.43%
Cache-misses	136.351.011	71.43%
Branches	17.716.290.666	71.43%
Branch-misses	56.290.608	71.43%
L1-dcache-load-misses	350.449.855	71.43%
LLC-load-misses	Não suportado	—

Tabela 5. C Serial: Análise de Hotspots via gprof (CPU 1)

% Time	Self (s)	Cum. (s)	Calls	Self/Call	Tot/Call	Função
100.00	49.99	49.99	5.001	0.01	0.01	evolve
0.00	49.99	0.00	1	0.00	49.99	game

Tabela 6. C Serial: Faltas de Cache por Instrução via Callgrind (CPU 1)

Função	Ir	Dr	Dw	I1mr	D1mr	D1mw	I1mr	DLmr
evolve	160,1Bi (99%)	17,5Bi (99%)	2,5Bi (93%)	9	78,3Mi (50%)	79,3Mi (50%)	9	5.001

Tabela 7. C Serial: Top 3 Chamadas de Sistema via strace (CPU 1)

% Time	Segundos	µs / Call	Calls	Syscall
13.04	0.000075	9	8	mmap
1.91	0.000011	3	3	fstat
0.70	0.000004	1	3	brk

Tabela 8. C Serial: Métricas de Hardware via perf stat (CPU 2)

Métrica	Valor	% Eventos
Cycles	91.783.122.923	—
Instructions	160.957.891.471	—
Cache-references	311.798.970	—
Cache-misses	921.887	—
Branches	17.653.280.461	—
Branch-misses	87.288.195	—
L1-dcache-load-misses	323.344.539	—
LLC-load-misses	129.138	—

Tabela 9. C Serial: Análise de Hotspots via gprof (CPU 2)

% Time	Self (s)	Cum. (s)	Calls	Self/Call	Tot/Call	Função
100.00	22.08	22.08	5.001	0.004	0.004	evolve
0.00	0.00	22.08	1	0.00	22.08	game

Tabela 10. C Serial: Faltas de Cache por Instrução via Callgrind (CPU 2)

Função	Ir	Dr	Dw	I1mr	D1mr	D1mw	I1mr	DLmr
evolve	160,1Bi (99%)	17,5Bi (99%)	2,5Bi (93%)	9	78,3Mi (50%)	79,3Mi (50%)	9	0

Tabela 11. C Serial: Top Chamadas de Sistema via strace (CPU 2)

% Time	Segundos	μs / Call	Calls	Errors	Syscall
0,00	0,000000	0	8	–	mmap
0,00	0,000000	0	3	–	fstat
0,00	0,000000	0	3	–	mprotect
0,00	0,000000	0	3	–	brk

Análise Python Serial

Tabela 12. Python Serial: Métricas de Hardware via perf stat (CPU 1)

Métrica	Valor	% Eventos
Cycles	5.313.323.064.534	71.43%
Instructions	17.591.544.342.430	71.43%
Cache-references	9.139.072.880	71.43%
Cache-misses	408.146.708	71.43%
Branches	3.034.443.352.177	71.43%
Branch-misses	7.839.717.447	71.43%
L1-dcache-load-misses	4.491.154.916	71.43%

Tabela 13. Python Serial: Top Hotspots Nativos via cProfile (CPU 1)

ncalls	tottime	percall	cumtime	percall	Função (arquivo: linha)
1	0.000	0.000	1579.484	1579.484	gol.py:1 (<module>)
1	0.001	0.001	1579.484	1579.484	gol.py:69 (main)
1	2.207	2.207	1579.483	1579.483	gol.py:53 (game)
5001	1577.275	0.315	1577.275	0.315	gol.py:14 (evolve)
1	0.000	0.000	0.001	0.001	gol.py:44 (count_alive)

Tabela 14. Python Serial: Top 3 Chamadas de Sistema via strace (CPU 1)

% Time	Segundos	µs / Call	Calls	Errors	Syscall
99.32	0.422889	5	80.049	–	brk
0.03	0.000119	1	78	24	newfstatat
0.07	0.000305	4	75	–	rt_sigaction

Tabela 15. Python Serial: Métricas de Hardware via perf stat (CPU 2)

Métrica	Valor	% Eventos
Cycles	4.460.144.195.537	–
Instructions	17.583.597.025.135	–
Cache-references	1.061.058.006	–
Cache-misses	160.778.323	–
Branches	3.032.810.529.031	–
Branch-misses	2.214.944.594	–
L1-dcache-load-misses	2.184.631.801	–
LLC-load-misses	6.023.437	–

Tabela 16. Python Serial: Top Hotspots Nativos via cProfile (CPU 2)]

ncalls	tottime	percall	cumtime	percall	Função (arquivo:linha)
5001	1472.690	0.294	1472.690	0.294	gol.py:evolve

Tabela 17. Python Serial: Top 3 Chamadas de Sistema via strace (CPU 2)

% Time	Segundos	μs / Call	Calls	Errors	Syscall
98,93	0,064318	0	80.045	–	brk
0,27	0,000178	2	78	24	newfstatat
0,19	0,000125	8	14	–	getdents64

4.4. Multithreading vs. Multiprocessing e o Impacto do GIL

Enquanto em C (via OpenMP) o *multithreading* é a abordagem padrão para paralelismo, no Python (CPython) essa estratégia esbarra no GIL (*Global Interpreter Lock*). O GIL é uma trava de exclusão mútua (*mutex*) que permite que apenas uma única *thread* execute *bytecodes* por vez, protegendo o gerenciamento automático de memória da linguagem contra condições de corrida.

Para algoritmos estritamente *CPU-bound* como o Jogo da Vida, o GIL anula o paralelismo real. Mesmo com múltiplas *threads* instanciadas, elas não calculam a matriz simultaneamente nos núcleos da CPU; apenas revezam a posse da trava. O *profiling* comprova este gargalo: o sistema demonstra características de *lock-bound*, perdendo tempo massivo em chamadas de bloqueio (*acquire_lock*, *sem_wait*) e trocas de contexto, o que frequentemente resulta em um desempenho inferior ao da versão serial.

A Tabela 19 evidencia empiricamente a serialização imposta pelo GIL. A função *process_chunk* registrou 1467s de tempo isolado (*tottime*), o que equivale a aproximadamente 0,073s por chamada. O somatório de quatro chamadas ($\sim 0,29s$) espelha quase perfeitamente os 0,316s gastos por geração na variante serial nativa. As funções de espera, como *Thread.join* e *Condition.wait*, acumulam milhares de segundos, indicando extrema ociosidade das *threads*.

Tabela 18. Python Multithreading: Principais Funções de Sincronização via cProfile (CPU 1)

Função	Chamadas	Tottime (s)	Cumtime (s)
Thread.start	4	0,000	0,020
Thread.join	4	0,000	1664,172
Thread.bootstrap_inner	4	14,334	1664,172
Lock.acquire	81449 / 41461	14,779	16,437
Condition.wait	20707	0,373	3354,980
Future.result	4001	20,167	3362,876

Para contornar o GIL e explorar múltiplos núcleos, o Python exige a adoção do *Multiprocessing*. Essa abordagem cria processos independentes no nível do sistema operacional, cada qual com seu próprio interpretador, espaço de memória e seu próprio GIL.

O *profiling* atesta que o custo inicial de instanciação (`posix.fork` e criação do `Pipe`) é irrisório, na casa dos 7 milissegundos. Adicionalmente, por utilizar `Process` e `fork` nativos em substituição ao `Pool.map`, o algoritmo contorna o custo pesado da serialização inicial (`pickle.dumps`).

Tabela 19. Python Multiprocessing: Custo de Criação e Comunicação (Lado do Pai) CPU 1

Função	Versão Pipe		Versão Shared Memory	
	Chamadas	Cumtime (s)	Chamadas	Cumtime (s)
Criação e Gestão de Processos				
<code>Process.start</code>	4	0,007	4	0,008
<code>posix.fork</code>	4	0,005	4	0,005
<code>Process.join</code>	4	0,001	4	0,002
Comunicação e Sincronização				
<code>Pipe()</code>	12	0,013	—	—
<code>Connection.recv (Pipe recv)</code>	4	203,074	—	—
<code>SharedMemory.__init__</code>	—	—	1	0,001
<code>Barrier()</code>	—	—	1	0,008
<code>Barrier.wait</code>	—	—	5.001	262,009
Serialização				
<code>marshal.loads / pickle</code>	30	0,004	41	0,005

Nota: Os tempos cumulativos extremos de `Connection.recv` (203,074s) e `Barrier.wait` (262,009s) englobam todo o tempo bloqueante de espera do processo mestre pelos cálculos dos workers.

O custo (*trade-off*) dessa solução é a perda do compartilhamento nativo de memória. O gargalo deixa de ser a criação de instâncias e passa a ser o alto custo da Comunicação Interprocessos (IPC). Conforme a Tabela 21, a versão via *Pipe* gasta quase a totalidade do seu tempo ocioso em `Connection.recv` (mais de 200 segundos), demonstrando forte característica de restrição por comunicação (*communication-bound*). Já na versão *Shared Memory*, embora a cópia via *Pipe* seja extinta, o *overhead* migra para a sincronização manual estrita: o sistema trava massivamente aguardando barreiras (`Barrier.wait`) e aquisição de semáforos (`SemLock.acquire`).

4.5. Multiprocessing: Comunicação via *Pipe* vs. Memória Compartilhada

Ao contornar o GIL por meio de processos isolados, o desafio do paralelismo em Python desloca-se para a forma como esses processos sincronizam os dados. O projeto contrasta duas estratégias fundamentais: a troca de mensagens (*Pipes*) e o acesso direto a recursos (*Shared Memory*).

A comunicação via *Pipes* opera por passagem explícita de mensagens. Para atualizar as "colunas fantasmas", o Python é obrigado a serializar e desserializar (*pickling*) suas estruturas de dados nativas a cada geração simulada. Esse ciclo constante de empacotamento impõe um *overhead* massivo de cópia de dados via Sistema Operacional,

Tabela 20. Python Multiprocessing: Custo de Criação e Comunicação (Lado do Pai) CPU 2

Função	Versão Pipe		Versão Shared Memory	
	Chamadas	Cumtime (s)	Chamadas	Cumtime (s)
Criação e Gestão de Processos				
<code>Process.start</code>	4	0,010	4	N/D
<code>posix.fork</code>	4	0,009	4	N/D
<code>Process.join</code>	4	0,001	4	N/D
Comunicação e Sincronização				
<code>Pipe()</code>	12	0,020	–	–
<code>Connection.recv (Pipe recv)</code>	4	257,458	–	–
<code>SharedMemory.__init__</code>	–	–	1	N/D
<code>Barrier()</code>	–	–	1	N/D
Serialização				
<code>marshal.loads / pickle</code>	30	0,003	N/D	N/D

Nota: O tempo cumulativo extremo de `Connection.recv` (257,458s) engloba todo o tempo bloqueante de espera do processo mestre pelos cálculos dos workers.

Tabela 21. Python Multiprocessing: Gargalos de Comunicação e Sincronização CPU 1

Versão	Função	Chamadas	Totttime (s)	Cumtime (s)	% do Total Ocioso
Pipe	<code>Connection.recv</code>	40.000	203,074	203,074	99,98%
	<code>posix.read</code>	8	203,073	203,073	99,98%
Shm	<code>Barrier.wait</code>	5.001	0,061	262,009	99,98%
	<code>Condition.wait</code>	9.053	0,075	261,769	99,89%
	<code>SemLock.acquire</code>	20.821	261,637	261,637	99,84%
	<code>SemLock.release</code>	27.159	0,052	0,052	0,02%

Tabela 22. Perfil de execução das principais funções do Python Multithreading.

Função / Método	Tempo Exclusivo (CPU 1)	Tempo Cumulativo (CPU 1)	Tempo Exclusivo (CPU 2)	Tempo Cumulativo (CPU 2)
base.py:422 (result)	20,167s	3.362,876s	0,092s	2.308,136s
threading.py:337 (wait)	0,373s	3.354,980s	0,256s	2.305,496s
queue.SimpleQueue (get)	—	—	193,457s	1.469,461s
threading.py:1097 (join)	0,000s	1.664,172s	0,000s	1.405,403s
thread.lock (acquire)	14,779s	16,437s	36,053s	36,495s
threading.py:975 (start)	0,000s	0,020s	0,000s	0,097s

transformando o gargalo da aplicação, que deixa de focar na lógica celular (*CPU-bound*) e passa a ser estrangulada pelo tráfego de dados (*I/O-bound*).

Em contrapartida, o uso de *Shared Memory* aloca um bloco contíguo de memória RAM acessível simultaneamente por todos os trabalhadores. Essa abordagem elimina totalmente a penalidade de serialização, recuperando o perfil computacional do algoritmo. O custo (*trade-off*) dessa eficiência reside na complexidade de implementação: abandona-se a flexibilidade das listas nativas em favor de *buffers* unidimensionais de *bytes*. Essa restrição estrutural exige cálculos manuais de índices e barreiras estritas de sincronização, mas possibilita a aplicação de otimizações de baixo nível, como as operações bit a bit em tabela de consulta (*lookup table*).

4.6. Análise de Escalabilidade e Eficiência Paralela

4.6.1. C Paralelizado com OpenMP

Tabela 23. Escalabilidade do C Paralelizado com OpenMP.

Trabalhadores	Tempo CPU1	Tempo CPU2	Speedup CPU1	Speedup CPU2	Eficiência CPU1	Eficiência CPU2
1 Thread	53,22s	22,99s	0,95x	0,96x	95,3%	96,1%
2 Threads	26,32s	12,42s	1,93x	1,78x	96,3%	88,9%
4 Threads	13,34s	7,90s	3,80x	2,80x	95,0%	69,9%
8 Threads	14,62s	8,46s	3,47x	2,61x	43,4%	32,6%

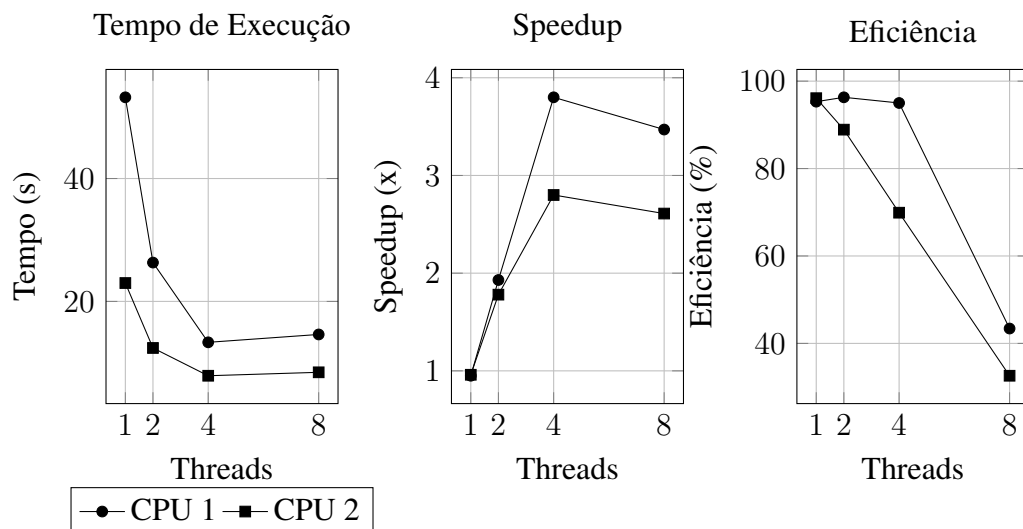


Figura 1. Tempo de execução, speedup e eficiência do C paralelizado com OpenMP.

4.6.2. Python Paralelo com Multithreading (ThreadPoolExecutor)

Tabela 24. Escalabilidade do Python Paralelo com Multithreading (ThreadPoolExecutor).

Trabalhadores	Tempo CPU1	Tempo CPU2	Speedup CPU1	Speedup CPU2	Eficiência CPU1	Eficiência CPU2
1 Thread	20m 51s	20m 54s	1,01x	0,96x	100,7%	95,8%
2 Threads	22m 47s	19m 37s	0,92x	1,02x	46,1%	51,0%
4 Threads	22m 35s	19m 13s	0,93x	1,04x	23,2%	26,0%
8 Threads	23m 53s	19m 15s	0,88x	1,04x	11,0%	13,0%

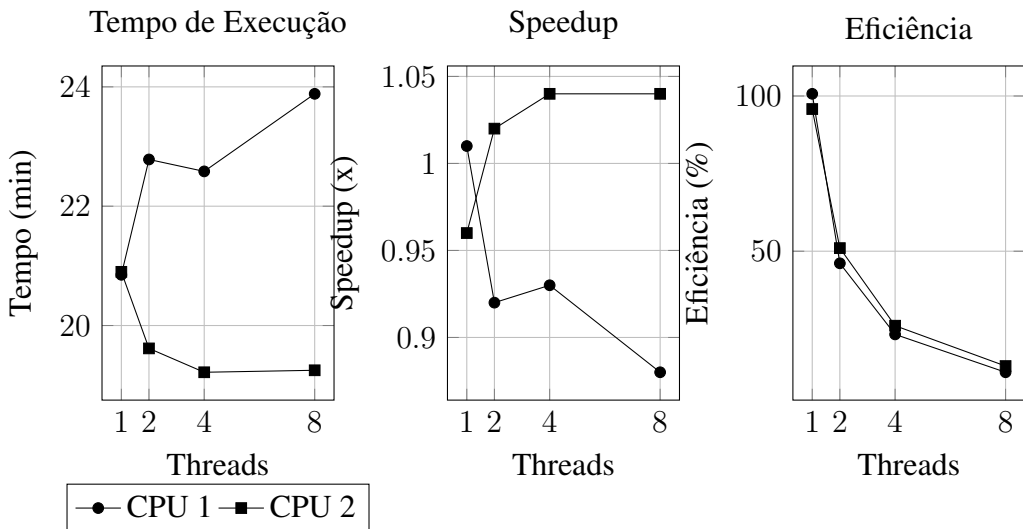


Figura 2. Tempo de execução, speedup e eficiência do Python com Multithreading.

4.6.3. Python Paralelo com Multiprocessing (Pipe)

Tabela 25. Escalabilidade do Python Paralelo com Multiprocessing utilizando Pipe.

Trabalhadores	Tempo CPU1	Tempo CPU2	Speedup CPU1	Speedup CPU2	Eficiência CPU1	Eficiência CPU2
1 Processo	11m 32s	12m 59s	1,82x	1,54x	182,0%	154,1%
2 Processos	05m 48s	07m 32s	3,61x	2,66x	180,5%	132,9%
4 Processos	03m 19s	04m 59s	6,30x	4,02x	157,6%	100,4%
8 Processos	03m 57s	04m 40s	5,32x	4,29x	66,4%	53,6%

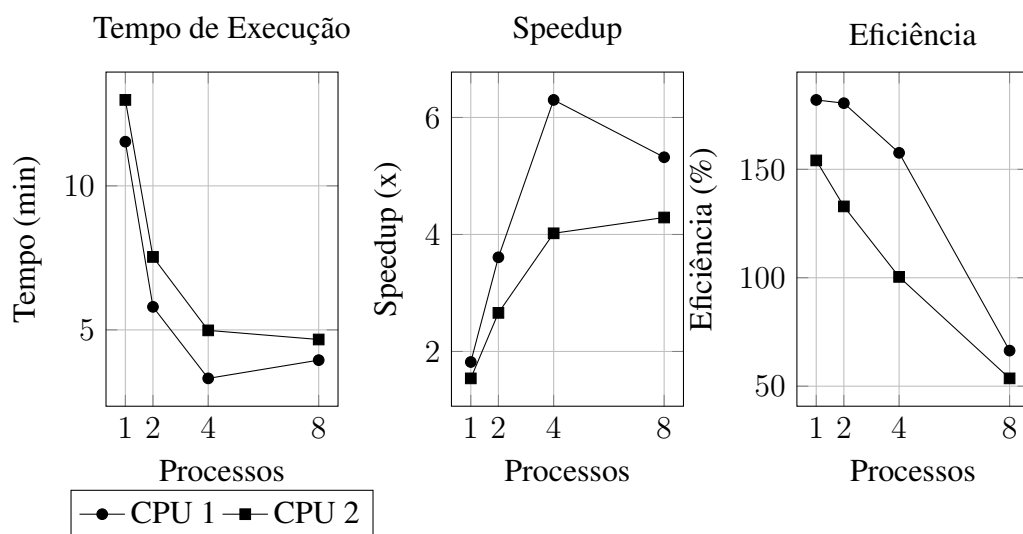


Figura 3. Tempo de execução, speedup e eficiência do Python com Multiprocessing utilizando Pipe.

4.6.4. Python Paralelo com Multiprocessing (Shared Memory)

Tabela 26. Escalabilidade do Python Paralelo com Multiprocessing utilizando Shared Memory.

Trabalhadores	Tempo CPU1	Tempo CPU2	Speedup CPU1	Speedup CPU2	Eficiência CPU1	Eficiência CPU2
1 Processo	14m 43s	13m 46s	1,43x	1,45x	142,5%	145,4%
2 Processos	09m 20s	07m 29s	2,25x	2,67x	112,4%	133,7%
4 Processos	04m 12s	05m 30s	4,99x	3,64x	124,7%	91,0%
8 Processos	04m 32s	04m 29s	4,63x	4,46x	57,8%	55,8%

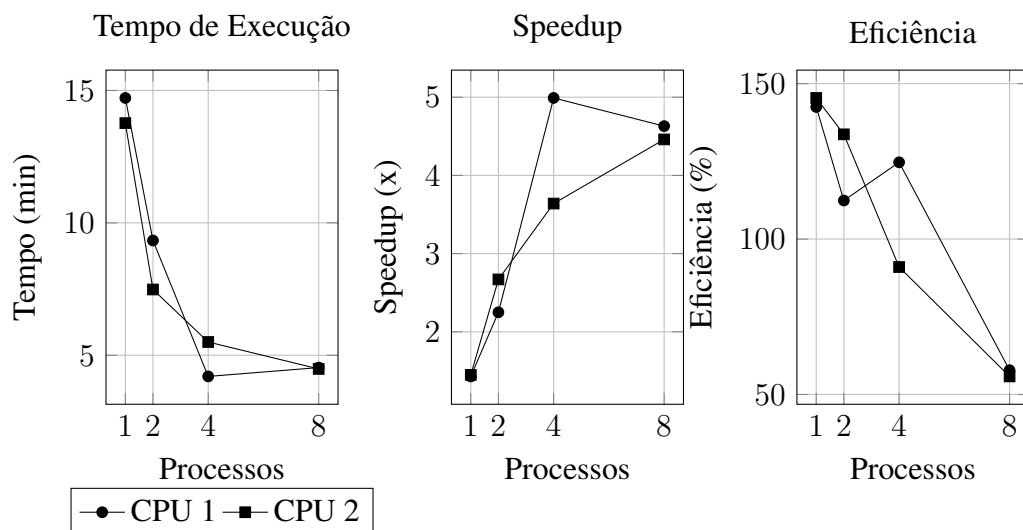


Figura 4. Tempo de execução, speedup e eficiência do Python com Multiprocessing utilizando Shared Memory.

4.7. Comparação Geral dos Modelos de Paralelização

Tabela 27. Tabela Geral de Desempenho (CPU 1) – /usr/bin/time -v

Implementação	Wall-clock	User Time (s)	Sys Time (s)	CPU %	Max RSS (KB)	Major PF	Minor PF	Vol. CS	Invol. CS
C Serial	0:50.71	50.66	0.01	99%	3.752	0	569	1	1.371
C OpenMP (4T)	0:13.18	52.69	0.01	399%	4.112	4	584	6	345
Python Serial	20:59.94	1254.30	5.33	99%	13.960	0	2.397.447	2	19.752
Python Thread (4T)	22:25.01	1344.69	10.26	100%	18.600	2	3.611	1.322.524	21.841
Python MP Pipe (4P)	3:33.75	834.86	0.97	391%	19.264	1	9.595	15.859	7.915
Python MP Shm (4P)	4:20.09	979.16	0.79	376%	20.544	13	8.736	64.538	8.363

Tabela 28. Tabela Geral de Desempenho (CPU 2) – /usr/bin/time -v

Implementação	Wall-clock	User Time (s)	Sys Time (s)	CPU %	Max RSS (KB)	Major PF	Minor PF	Vol. CS	Invol. CS
C Serial	0:22.10	22.12	0.00	99%	3.960	0	570	1	709
C OpenMP (4T)	0:07.90	31.60	0.00	399%	4.252	0	589	1	112
Python Serial	20:01.01	1198.54	2.42	99%	13.992	0	2.397.003	1	10.048
Python Thread (4T)	19:13.51	1155.78	3.06	100%	18.852	0	3.167	884.100	7.288
Python MP Pipe (4P)	4:59.03	1158.72	0.53	387%	19.140	0	9.161	16.227	5.707
Python MP Shm (4P)	5:30.80	1230.99	0.46	372%	20.312	0	8.315	63.930	8.383

Abordagens Paralelas e Escalabilidade

Tabela 29. C OpenMP (4 Threads): Métricas de Hardware e Concorrência (CPU 1)

Métrica	Valor	% Eventos
Cycles	221.902.421.354	71.44%
Instructions	174.652.053.422	71.44%
Cache-misses	252.405.076	71.42%
Branch-misses	59.680.124	71.42%
Context-switches	311	—
CPU-migrations	3	—

Tabela 30. Python Multiprocessing (4 Processos): IPC e Instruções (CPU 1)

Abordagem	Métrica	Valor	% Eventos	
Pipe	Cycles	2.816.993.707.354	71.43%	
	Instructions	9.910.640.466.965	71.43%	<i>*Nota: Os perfis de</i>
Shared Mem	Cycles	3.520.700.946.895	71.43%	
Shared Mem	Instructions	10.895.935.969.543	71.43%	

Multithreading e detalhes de I/O em cProfile e perf report não retornaram gargalos identificáveis no profiler desta máquina.

Tabela 31. Dados Brutos de Escalabilidade ($p = 1, 2, 4, 8$) (CPU 1)

Implementação	Núcleos/Processos	Tempo (Wall-clock)
C OpenMP	1	0:53.22
C OpenMP	2	0:26.32
C OpenMP	4	0:13.34
C OpenMP	8	0:14.62
Python Multithreading	1	20:51.45
Python Multithreading	2	22:47.47
Python Multithreading	4	22:35.73
Python Multithreading	8	23:53.71
Python MP (Pipe)	1	11:32.42
Python MP (Pipe)	2	5:48.94
Python MP (Pipe)	4	3:19.87
Python MP (Pipe)	8	3:57.04
Python MP (Shared Mem)	1	14:43.94
Python MP (Shared Mem)	2	9:20.43
Python MP (Shared Mem)	4	4:12.61
Python MP (Shared Mem)	8	4:32.41